

2026 年 5 月 13 日 Version x.x

ハッカソン ～計画書と設計書～

チーム 2

24G1146 : 川又 快斗

24G1053 : 熊本 朱純

24G1133 : 安尾 晃貴

25G1084 : 陳 夏薇

25G1089 : 常世田 隆一

25G1110 : 早川 和輝

25G1115 : 福島 健一

目次

第 1 章	システムの基本設計	5
1.1	機能一覧	5
1.2	システム構成図	6
1.3	操作インターフェース設計	8
1.4	システムのシーケンス図	9
第 2 章	システムの詳細設計	11
2.1	部品一覧	11
2.2	ハードウェア設計	11
2.2.1	センサ制御ユニット	11
2.2.2	同期制御ユニット	14
2.2.3	楽器制御ユニット	14
2.3	ソフトウェア設計	15
2.3.1	ファイル構成	15
2.3.2	オブジェクト設計 (クラス図)	17
2.3.3	関数設計	20
2.4	処理フローの設計	23
2.5	テストの設計	24

第 1 章 システムの基本設計

本章では、指揮棒型入力デバイスを用いて複数の Arduino と Processing を連携させる自動合奏システムの基本設計について述べる。本システムは、センサ制御、同期制御、楽器制御の三つの役割に機能を分割した分散構成を採用しており、ユーザの指揮動作から演奏開始判定、テンポ更新、音声出力、視覚演出までを一連の処理として実現する。

1.1 機能一覧

本システムの主要機能を表 1.1 に示す。センサ制御部で指揮動作を取得し、同期制御部が開始信号とテンポ情報を分配し、楽器制御部と音声合成部が発音と再生を担当する構成である。

表 1.1 機能一覧

機能名	概要
動作入力検知機能	指揮棒型デバイスに搭載した IMU センサから加速度および角速度を取得し、演奏開始の合図となるダウンビートを検知する。
テンポ算出・更新機能	指揮動作の拍間隔から BPM を算出し、演奏中のテンポを段階的に更新する。
分散型同期通信機能	センサ制御部で判定した START 信号およびテンポ情報を同期制御部が受信し、各楽器制御ユニットへ分配する。
自動演奏機能	各楽器制御ユニットが受信した情報に基づいて演奏タイミングを決定し、Processing へ周波数情報を送信する。
視覚演出機能	楽器制御ユニットに接続した LED マトリクスおよびサーボモータを用いて、発音と同時に視覚的フィードバックを行う。

表 1.1 は、センサ制御、同期制御、楽器制御、音声合成の各層で実行する処理を整理した一覧である。開始判定とテンポ更新はセンサ制御部が担い、開始指示とテンポ配信は同期制御部が担い、発音タイミングの管理と視覚演出は楽器制御部が担う。

動作入力検知機能は、センサ制御部の中でも最も上流に位置する機能であり、以降の同期制御および楽器制御の成否を左右する。そのため、単純に加速度の大小だけで判定するのではなく、前回の拍からの経過時間や角速度の変化も合わせて参照し、不適切な誤検出を抑えながら有効な指揮動作のみを抽出する構成とする。

テンポ算出・更新機能では、単発のピーク間隔をそのままテンポ値として採用するとゆらぎが大きくなるため、複数拍分の値を蓄積し、小節単位または一定区間単位で平均化した BPM を用いる。これにより、指揮者の動きに自然に追従しつつも、演奏系に過剰な揺れを伝えない安定したテンポ制御が可能となる。

分散型同期通信機能では、センサ制御部が生成した抽象的な演奏開始情報やテンポ情報を、同期制御部が各楽器制御ユニットの実行可能な命令へ変換する。この中継を設けることで、センサ制御部は動作解析に集中でき、楽器制御部は受信した共通形式の命令だけを解釈すればよい構造になる。

各機能の内部処理は、センサ制御部の IMU 取得、前処理、ピーク検出、ダウンビート検出、BPM 算出、テンポ段階化、送信処理に分かれる。同期制御部は受信情報の保持、演奏開始判定の共有、ポーリング応答、テンポ更新値の配信、担当楽器への順序制御を担う。楽器制御部は待機時間計算、周波数送信、LED 表示更新、サーボ駆動を担う。

この一覧は、センサ制御、同期制御、楽器制御、音声合成の責務分担をそのまま示している。各機能の境界を保ちながら、処理をクラス、関数、通信手順へ落とし込む。

1.2 システム構成図

本システムは、指揮棒型デバイスに搭載された IMU センサ、センサ制御用 Arduino 1 台、同期制御用 Arduino 1 台、楽器制御用 Arduino 4 台、および音声処理用 PC で構成される。全体の構成図を図 1.1 に示す。

図 1.1 は、指揮棒型デバイス、センサ制御部、同期制御部、複数の楽器制御ユニット、Processing 実行 PC の接続関係を示す図である。指揮動作はセンサ制御部で解析され、START 信号とテンポ情報として同期制御部へ渡される。同期制御部はそれらを各楽器制御ユニットへ配信し、楽器制御ユニットは発音情報を Processing へ送信する。

センサ制御部は、IMU センサから取得した加速度および角速度を前処理し、拍検出、ダウンビート判定、BPM 算出を行う役割を担う。演奏開始が成立した場合は

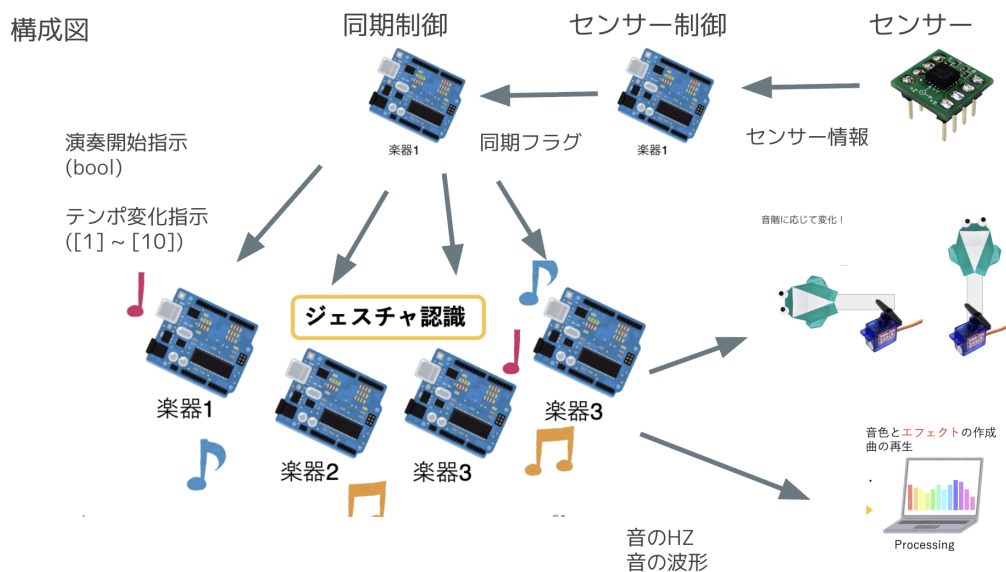


図 1.1 システム構成図

START 信号を、同一演奏中にテンポの変化が検出された場合は BPM 情報を同期制御部へ送信する。

同期制御部は、センサ制御部から受信した情報を全体の演奏タイミングへ変換する中継ユニットである。各楽器制御ユニットからの状態要求に応答しつつ、開始指示やテンポ更新値を適切なタイミングで配信することで、輪唱に必要な通信順序と同期性を維持する。

楽器制御部は、リズム用 3 台とビート用 1 台に分かれ、受信した同期情報に従って独立に発音タイミングを管理する。各ユニットは Processing へ周波数または演奏トリガを送信し、さらに一部のユニットではサーボモータおよび LED マトリクスを用いた視覚演出も同時に制御する。

情報の流れとしては、まずユーザの指揮動作がセンサとして観測され、その観測結果が演奏を開始するための開始指示または演奏速度を変化させるためのテンポ情報へ変換される。次に、同期制御部がそれらを各楽器制御ユニットへ配信し、楽器制御ユニットは個別の楽曲データに従って発音時刻を生成する。最後に、Processing が送られてきた周波数情報を音声へ変換し、物理演出系がそれに同期して動作することで、入力から音と動きまでが一貫した流れとして接続される。

また、本システムでは制御を 1 台のマイコンへ集中させず、役割別に分散させているため、機能追加や調整時の影響範囲を限定できる利点がある。例えば、センサ判定の閾値や BPM 算出方法を変更しても、楽器制御側の楽曲データや音声合成の構成を

大きく変更せずに済む。同様に、楽器ユニットを追加する場合でも、同期制御部の配信対象を拡張することで対応しやすい構成となっている。

図 1.2 は、センサ制御部のハードウェア構成を示したものである。指揮棒に装着した IMU センサ、Arduino UNO R4 WiFi、および I2C 接続の関係を示し、配線と信号の流れを確認できるようにしている。

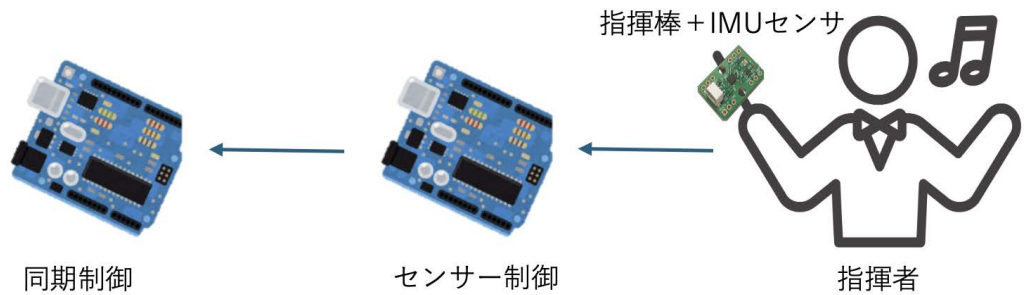


図 1.2 センサ制御部のハードウェア構成

1.3 操作インターフェース設計

本システムの主な操作インターフェースは、ユーザが手に持つ指揮棒型デバイスである。一般的なボタン操作ではなく、指揮動作そのものを入力として扱うことで、直感的な演奏制御を実現する。主な操作内容を表 1.2 に示す。

表 1.2 操作インターフェース

インターフェース	操作内容	システム動作
指揮棒型デバイス	ユーザが振る	IMU センサが加速度および角速度を取得し、センサ制御部が動作解析を行う。
指揮開始動作	初回の強い振り下ろしを行う	ダウンビートが成立した時点で START 信号を生成し、同期制御部へ送信する。
指揮速度の変更	指揮棒の往復周期を変える	拍間隔から BPM を再算出し、同期制御部を介して楽器制御部の演奏速度を更新する。

表 1.2 は、ユーザの身体動作とシステムの動作を対応づけた一覧である。指揮棒を振る操作は IMU データ取得に、初回の強い振り下ろしは START 信号生成に、往復周期の変化は BPM 更新に対応する。

このように、ユーザは指揮棒を振るだけで演奏開始とテンポ変更の双方を行える。入力手段を単一の物理デバイスへ集約することで、システム全体の操作性と演出性を両立している。

操作インターフェースを単純化する理由は、発表や展示の場面において、ユーザが短時間で操作方法を理解できるようにするためである。複数のボタンや設定値を用意すると、開始動作とテンポ変更の切り替えが煩雑になるが、本システムでは指揮動作の種類と周期のみで制御を完結させるため、操作学習の負担が小さい。

さらに、このインターフェースは単に使いやすいだけでなく、システムの見た目と機能を一致させる役割も果たしている。ユーザが指揮棒を振るという行為そのものが演奏制御の入力となるため、観客にとっても「どの操作がどの結果を生んでいるか」が理解しやすく、展示システムとしての説明性にも優れている。

1.4 システムのシーケンス図

システム全体の通信手順を図 1.3 に示す。

状態遷移図

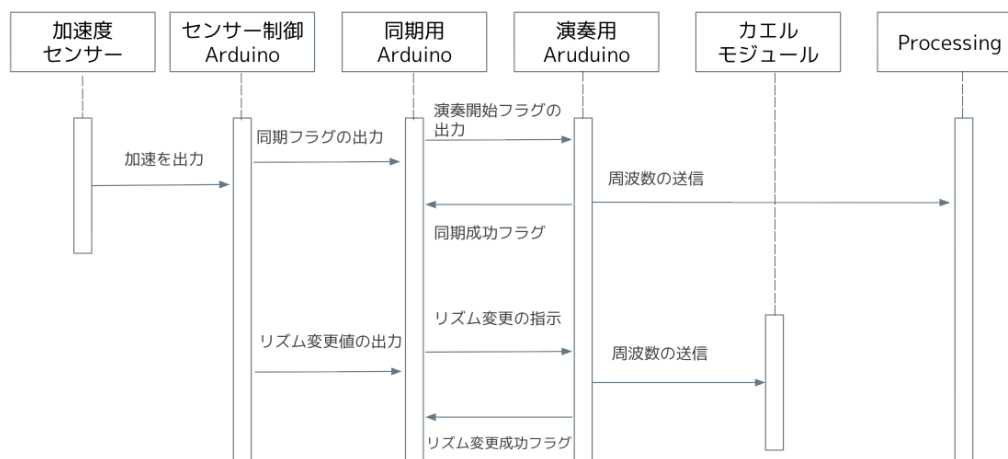


図 1.3 システムのシーケンス図

図 1.3 は、センサ制御部から同期制御部、楽器制御ユニット、Processing へ信号が流れる順序を示す図である。START 信号の生成、演奏開始指示の配信、テンポ更新の通知、周波数情報の送信が時系列で表されている。

システム起動後、センサ制御部は待機状態で IMU データを監視する。有効な指揮開始動作が検出されると開始判定状態へ移行し、拍検出およびダウンビート判定を行う。開始条件が成立した場合、センサ制御部は START 信号を同期制御部へ送信する。

同期制御部は START 信号を受信すると、各楽器制御ユニットへ演奏開始指示を配信する。各楽器制御ユニットは指示を受けると演奏準備状態から演奏状態へ遷移し、Processing への周波数送信および視覚演出の制御を開始する。必要に応じて、演奏開始可否や最新状態を同期制御部へ返すことで、全体の同期を確認する。

演奏中は、センサ制御部が継続的に指揮動作を監視し、拍間隔の変化からテンポ更新値を算出する。更新値は同期制御部へ送信され、同期制御部は各楽器制御ユニットへ新しいテンポを通知する。これにより、演奏中のテンポ変更にも追従できる状態遷移となっている。

状態遷移の設計において重要なのは、開始判定とテンポ更新を同じ入力デバイスで扱いながら、両者を誤って混同しないことである。そのため、本システムでは初回の有効なダウンビート成立を特別なイベントとして扱い、演奏開始済みフラグの状態によって送信内容を切り替える。これにより、演奏開始前は START 信号のみを、演奏開始後はテンポ情報のみを優先して送ることができる。

また、同期制御部と楽器制御部の間に確認応答の概念を入れることで、一時的な通信遅延やタイミングのずれが起きても、少なくとも現在の演奏状態と最新テンポを各ユニットが共有しやすい構成となる。この設計は、輪唱のように複数ユニットが異なるタイミングで動作する場面で特に有効である。

第 2 章 システムの詳細設計

本章では、前章で示した基本設計に基づき、センサ制御、同期制御、楽器制御の各ユニットを具体的に実装するための詳細設計を示す。役割ごとに必要な部品、ハードウェア構成、ソフトウェア構成、処理フロー、テスト方法を整理し、分散制御システムとして整合するように設計する。

2.1 部品一覧

本システムで使用する主要部品を表 2.1 に示す。センサ制御部では指揮動作の取得と解析、同期制御部では信号分配、楽器制御部では演奏および視覚演出を担当するため、それぞれの役割に応じた部品を選定している。

表 2.1 は、実装に必要な物理部品を役割別に整理した一覧である。同じ Arduino UNO R4 WiFi でも、センサ制御用、同期制御用、楽器制御用で用途が異なるため、担当機能を併記している。IMU センサやサーボモータは、演奏制御と視覚演出を成立させる主要部品である。

部品候補としては、3 軸加速度センサの ADXL345 と 6 軸 IMU の MPU6050 がある。いずれも指揮棒の動作取得に利用できるため、本設計では「指揮棒に搭載される IMU センサ」として扱う。

指揮棒、ブレッドボード、ジャンパ線は、実機として構築する際に必要な試作部材である。

2.2 ハードウェア設計

2.2.1 センサ制御ユニット

センサ制御ユニットは、指揮棒型デバイスに取り付けた IMU センサと Arduino UNO R4 WiFi から構成される。IMU センサは I2C 通信を用いて Arduino へ接続し、加速度と角速度の 6 軸情報を周期的に取得する。Arduino は取得データに対してノ

表 2.1 主要部品一覧

部品名	数量	用途
Arduino UNO R4 WiFi (センサ制御用)	1 台	IMU センサから取得した加速度および角速度を処理し, START 信号と BPM 情報を生成する.
Arduino UNO R4 WiFi (同期制御用)	1 台	センサ制御部から受信した情報を各楽器制御ユニットへ分配し, 全体の演奏タイミングを管理する.
Arduino UNO R4 WiFi (楽器制御【リズム用】)	3 台	メロディを含む演奏データを保持し, Processing への周波数送信と視覚演出制御を行う.
Arduino UNO R4 WiFi (楽器制御【ビート用】)	1 台	打楽器的なビート演奏を担当し, 輪唱全体のリズムを補強する.
IMU センサモジュール	1 個	指揮棒に取り付け, 3 軸加速度および角速度を取得して指揮動作の解析に用いる.
SG92R (マイクロサーボモータ)	6 個	発音と連動して紙のカエルを起き上がらせる物理演出機構を構成する.
ブレッドボード・ジャンパ線	一式	各 Arduino およびセンサ, サーボモータの配線と回路試作に用いる.
PC (Processing 実行環境)	1 台	受信した周波数情報に基づいて音声波形を生成し, スピーカーへ出力する.

イズ除去や重力成分の影響低減を行い、拍のピーク検出とダウンビート判定を実施する。

I2C 接続では、電源線として VCC と GND、信号線として SDA および SCL を用いる。この構成は配線数が少なく、指揮棒型デバイスの小型化に向いている。また、センサ値を連続的に取得するため、読み取り周期は一定に保ち、サンプリング間隔のばらつきが BPM 算出へ影響しないようにする必要がある。

センサ制御ユニット内では、加速度データだけでなく角速度データも併用することで、単なる振動や持ち替え動作と、演奏開始を意図した振り下ろし動作とを区別しやすくしている。これにより、展示環境で生じやすい雑音的な入力を減らし、開始判定の信頼性を高める。

IMU センサの VCC 端子は Arduino の 5V 端子へ、GND 端子は GND へ、SDA 端子および SCL 端子は I2C 通信用端子へ接続する。この構成は配線数が少なく、試作と保守が容易である。

回路図としては、図 2.1 のように Arduino UNO R4 WiFi と MPU6050 を接続する構成を採る。MPU6050 の VCC を 5V、GND を GND、SDA と SCL を I2C 通信用の端子へ接続することで、センサ値を安定して取得できる構成になっている。

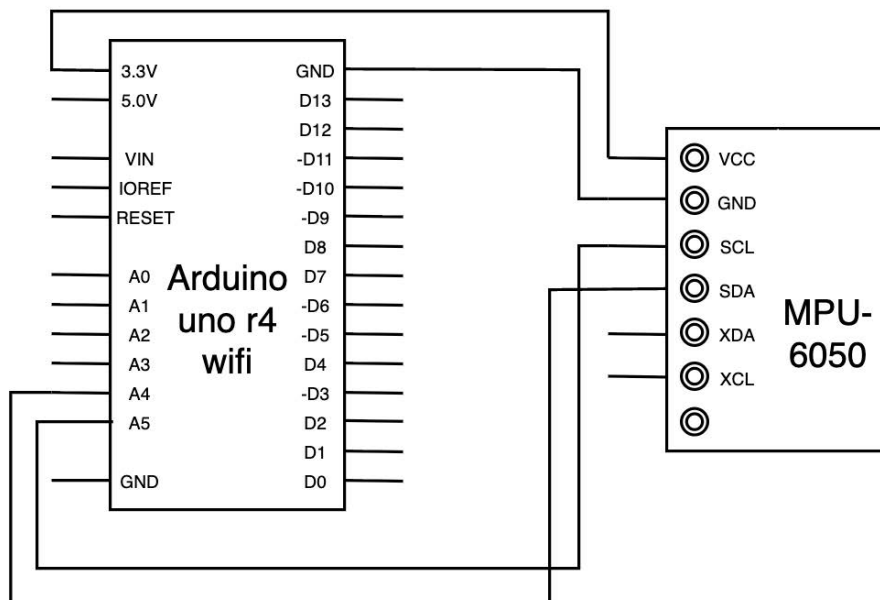


図 2.1 センサ制御部の回路図

図 2.1 は、センサ制御ユニット内で IMU センサを Arduino に接続する回路図であ

る。電源線と I2C 信号線の接続先を明示し、配線時の接続関係を確認できるようにしている。

2.2.2 同期制御ユニット

同期制御ユニットは、センサ制御部から送られてくる START 信号およびテンポ情報を受け取り、複数の楽器制御ユニットへ適切に分配する中継ユニットである。本ユニットはサーバー的な役割を担い、楽器制御ユニットからの状態確認要求に応答しながら、現在の演奏開始状態とテンポ情報を保持する。通信方式はポーリングを前提とし、楽器制御ユニットが継続的に指示を要求することで、通信状態を確認しつつ信号配信の確実性を高める。

同期制御ユニットを独立させることで、センサ制御部で得られた入力解析結果を整理し、楽器制御ユニットごとに必要な形式へ変換できる。演奏開始直後は全ユニットに同じ開始許可を与えつつ、各ユニットは内部に保持する楽曲データに基づいて異なるタイミングで動作する。中央の同期制御部は全体時刻の基準を提供し、各ユニットはその基準に従って自律動作する。

ポーリング方式を採用する理由は、各ユニットの通信タイミングを比較的単純に管理できるためである。同期制御ユニット側が一方向的にすべての命令を送る方式に比べて、各楽器制御ユニットが自ら状態を確認するため、受信側の準備状態に応じた安定した通信処理を組みやすい。

同期制御ユニットはサーバー、各楽器制御ユニットはクライアントとして扱う。クライアントからの問い合わせに応じてサーバーが現在の演奏状態を返し、各楽器制御ユニットは自分のタイミングで最新状態を取得する。

2.2.3 楽器制御ユニット

楽器制御ユニットは、リズム用 3 台とビート用 1 台の計 4 台で構成される。各ユニットは同期制御部から開始信号および BPM 情報を受信し、自身の楽曲データに基づいて発音タイミングを管理する。リズム用ユニットの一部にはサーボモータを接続し、音階に対応した紙のカエルの可動演出を行う。また、Arduino UNO R4 WiFi に搭載された LED マトリクスを用いて演奏状態を表示し、視覚的にも動作が分かる構成としている。

各楽器制御ユニットは、同じプログラム構成を持ちながら、保持する楽曲データの内容によって担当パートが分かれる構造とする。これにより、プログラムの再利用性

を高めつつ、メロディ担当とビート担当で異なる演奏パターンを実現できる。また、演奏データは周波数と音価の組で管理し、BPM 変更時には音価そのものではなく待機時間の計算式を更新することで、曲全体の比率を保ったままテンポだけを変化させる。

視覚演出では、サーボモータを音階ごとに対応づけることで、どの音が鳴ったかを物理的な動きとして観察できるようにしている。LED マトリクスは演奏状態の補助表示として利用し、システムが待機中か、演奏中か、あるいは発音直後かを視覚的に示すことで、デバッグ時にも有用な情報を提供する。

楽曲は「かえるの合唱」を対象とし、ド、レ、ミ、ファ、ソ、ラの6音階に対応した数のサーボモータを配置する。各サーボには異なる高さや色を持つ棒と紙のカエルを取り付け、該当音階の発音時にそのカエルだけが起き上がるようにする。

2.3 ソフトウェア設計

ソフトウェアは、センサ制御、同期制御、楽器制御、音声合成の四つの観点から設計する。センサ制御では動作解析とテンポ生成、同期制御では情報保持と分配、楽器制御では演奏タイミング管理と演出制御、音声合成では周波数データの再生を担当する。

全体の設計方針として、各制御単位は責務を明確に分離し、入力解析、命令分配、演奏制御、音声生成が互いに独立して保守できるようにする。特に、センサ処理はノイズや個人差の影響を受けやすいため、他の制御処理と密結合にせず、テンポや開始指示という抽象化された情報だけを外部へ渡す構造とする。この分離により、センサ側のアルゴリズム改善と楽器側の楽曲追加を並行して進めやすくなる。

2.3.1 ファイル構成

各制御系のファイル構成と役割を表 2.2 に示す。入力取得、前処理、判定、送信、音生成を別ファイルに分け、修正対象を局所化している。

表 2.2 は、実装ファイルを制御系ごとに分割した一覧である。入力取得、前処理、判定、送信、音生成の境界を明確にしている。

センサ制御側では、入力取得、前処理、拍検出、テンポ算出、送信処理を分離しているため、各処理の調整を局所的に行える。例えば、ピーク検出の閾値変更は BeatDetector 周辺だけで完結し、通信形式の変更は SensorSender 周辺のみを修正

表 2.2 制御に関連するファイル構成

プロジェクト	ファイル名	役割
センサ制御 (Arduino)	SensorMain.ino	IMU データ取得, 状態管理, START 信号およびテンポ情報送信の主処理を行う。
	IMUReader.h/cpp	IMU センサから 3 軸加速度および 3 軸角速度を取得する。
	MotionFilter.h/cpp	重力除去, 合成加速度算出, 平滑化処理を行う。
	BeatDetector.h/cpp	加速度ピークを検出し, 拍タイミングを判定する。
	DownbeatDetector.h/cpp	ジャイロデータからダウンビートを検出し, 小節開始位置を判定する。
	TempoCalculator.h/cpp	拍間隔から BPM を算出し, テンポレベルへ変換する。
	SensorController.h/cpp	センサ制御全体の状態管理と各処理クラスの呼び出しを統括する。
	SensorSender.h/cpp	START 信号およびテンポ情報を同期制御ユニットへ送信する。
同期制御 (Arduino)	SyncMain.ino	センサ制御部から受信した情報を保持し, 各楽器制御ユニットへ開始指示とテンポ更新値を配信する。
	SyncController.h/cpp	指示状態の更新, ポーリング応答, 配信順序の管理を行う。
楽器・演出制御 (Arduino)	InstrumentMain.ino	同期信号の受信, 演奏管理, 物理演出およびマトリクス表示の主処理を行う。
	MusicData.h	ユニットの役割に応じた周波数・タイミングデータを定義する。
	InstrumentController.h/cpp	待機時間計算, 演奏トリガ送信, LED マトリクス制御を実装する。
	FrogServoController.h/cpp	サーボモータを用いた紙のカエルの可動制御を実装する。
音声合成 (Processing)	SoundSynthesizer.pde	音声合成プログラム全体の制御を行う。
	SerialReceiver.pde	シリアル通信の受信と周波数データの解析を行う。
	AudioPlayer.pde	発振器を用いた音声波形の生成と出力を行う。

すればよい。

同期制御側はファイル数を必要最小限に抑え、状態保持と配信制御に特化させる。これは、同期制御部の責務を複雑にしすぎると、全体時刻の基準生成と通信処理が混在して不具合調査が難しくなるためである。

楽器制御側と Processing 側は、演奏トリガの送信までを Arduino 側、実際の音生成を PC 側へ分離している。この構成により、マイコンの処理負荷を抑えながら、より柔軟な音色生成や複数音の合成を Processing 側で扱いやすくしている。

センサ制御側には DownbeatDetector.h/cpp と SensorController.h/cpp が含まれる。前者はジャイロデータから小節開始に相当するダウンビートを判定し、後者は IMUReader, MotionFilter, BeatDetector, TempoCalculator, SensorSender を呼び出して全体状態を管理する。

2.3.2 オブジェクト設計 (クラス図)

主なクラス構成を表 2.3 に示す。センサ制御部では入力解析用クラス、同期制御部では配信用クラス、楽器制御部では演奏・演出制御用クラス、Processing 側では受信・再生用クラスを用いる。

表 2.3 は、システムの主要なオブジェクトを責務単位で整理したものである。センサ入力の取得から状態管理、通信、演出、音生成までをクラス単位で示している。

図 2.2 には、これらのクラスの依存関係と役割分担をまとめたクラス図を示す。センサ側のクラスが入力取得から判定までを順に担当し、同期制御や楽器制御のクラスがその結果を受けて動作する構造である。

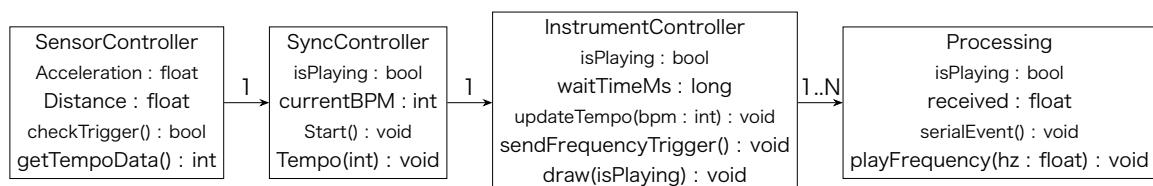


図 2.2 システムのクラス図

クラス間の依存関係は、データの流れに沿って一方向になるよう意識して設計する。センサ制御部では IMUReader が原データを供給し、MotionFilter が前処理を行い、BeatDetector および TempoCalculator が意味のある制御情報へ変換する。SensorSender はその結果だけを送信するため、入力取得の詳細を直接持たない。

表 2.3 クラス設計

クラス名	主な属性	主なメソッド
IMUReader	ax, ay, az, gx, gy, gz	readIMU()
MotionFilter	motion, threshold	removeGravity(), smooth(), calcMotion()
BeatDetector	lastPeakTime, peakFlag	detectPeak()
DownbeatDetector	lastDownbeatTime, downbeatFlag	detectDownbeat()
TempoCalculator	bpm, intervalBuf, level	calcBPM(), averageBPM(), convertLevel()
SensorSender	sendData	sendStart(), sendTempo()
SensorController	state, isStarted	update(), resetState()
SyncController	startFlag, tempoValue, requestState	updateCommand(), respondPolling()
InstrumentController	bpm, nextTick, playState	updateTempo(), sendFrequencyTrigger(), drawFrogMatrix()
FrogServoController	currentAngle, noteIndex	moveFrog()
SerialReceiver	receiveBuffer	serialEvent()
AudioPlayer	oscillator, frequency	playFrequency()

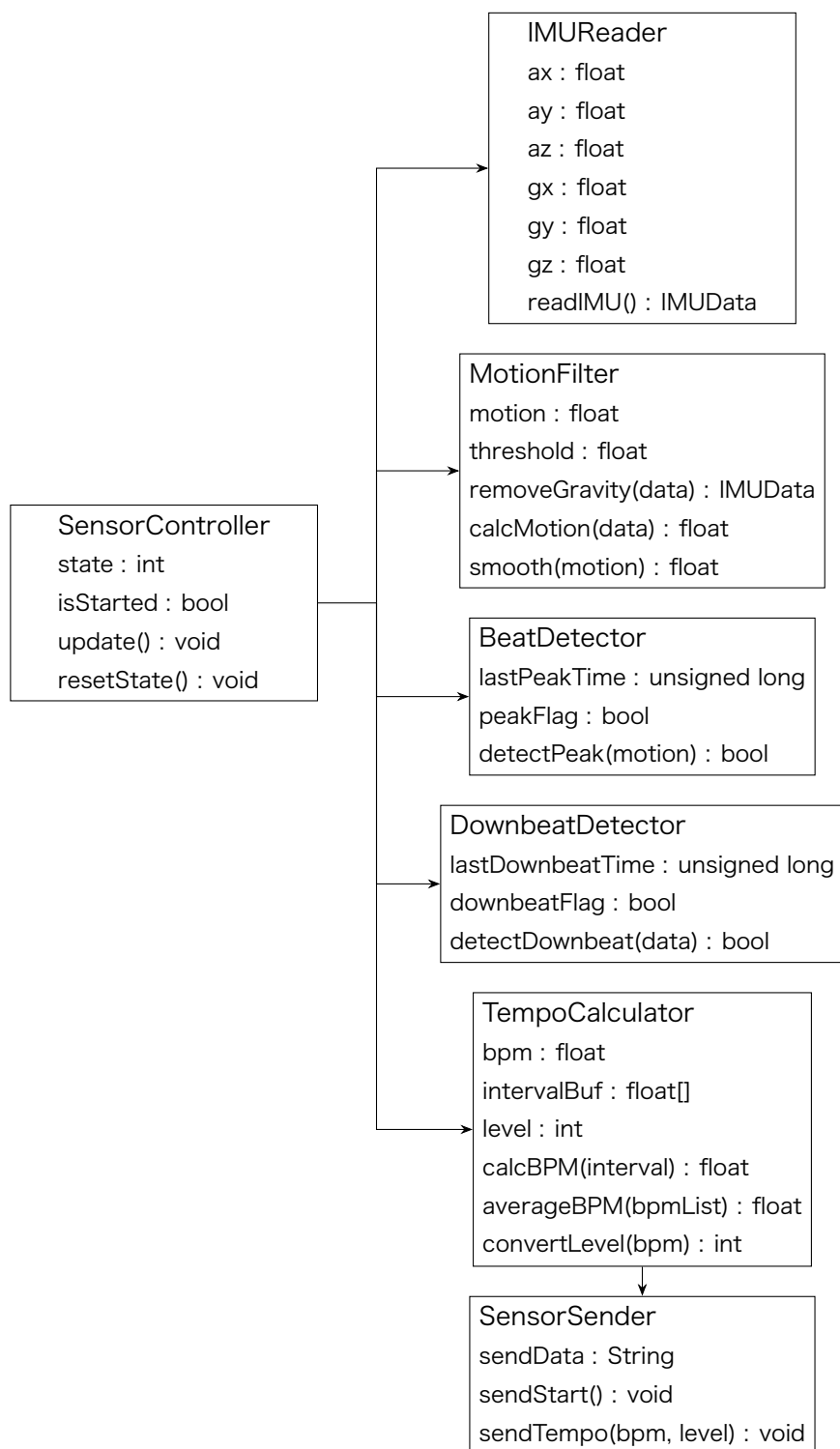


図 2.3 センサ制御部のクラス図

同期制御部では SyncController が中心となり、外部から受け取った開始状態とテンポ情報を内部に保持しつつ、楽器制御ユニットからの問い合わせへ応答する。楽器制御部では InstrumentController が演奏タイミングの中核を担当し、必要に応じて FrogServoController を呼び出して演出を同期させる。Processing 側では SerialReceiver と AudioPlayer が役割を分担し、通信と音生成を分離する。

センサ制御側のクラス構成は、IMUReader が原データを取得し、MotionFilter が前処理を担当し、BeatDetector と DownbeatDetector がイベント検出を行い、TempoCalculator がテンポ情報を算出し、SensorSender が外部送信を担当する一方向の流れである。上位では SensorController が状態変数を管理し、一連の処理順序を統括する。

2.3.3 関数設計

主要関数の仕様を表 2.4 に示す。センサ入力からテンポ生成、指示配信、発音制御、音声出力までの流れを関数レベルで対応づけている。

表 2.4 は、各クラスが持つ関数を処理単位で整理したものである。引数と戻り値に加え、前処理、イベント検出、通信、音生成の位置づけを確認できる。

これらの関数は、ループ処理の中で高い頻度で呼び出されるものと、イベント発生時にのみ呼び出されるものに分けられる。readIMU, detectPeak, calcBPM などは周期的に実行される基盤処理であり、sendStart や sendTempo は条件成立時のみ発火するイベント駆動型の処理である。この区別により、演奏中の不要な通信量を抑えつつ、必要なタイミングでは即座に状態を更新できる。

また、楽器制御系の関数は、受信した BPM に応じて内部の時間管理を更新する updateTempo と、実際に発音指示を外部へ送る sendFrequencyTrigger を分離している。この構成により、テンポ更新と発音実行の責務が明確になり、タイミング制御の誤差要因を切り分けやすい。

removeGravity, calcMotion, smooth, detectDownbeat, averageBPM, convertLevel は、前処理と判定精度の向上に関係する補助関数である。これらの処理が sendStart および sendTempo の精度に影響するため、関数設計では前段のデータ整形も含めて扱う。

センサ制御部単体の処理フロー図を図 2.4 に示す。初期化後に IMU データを取得し、前処理を施してからピーク検出とダウンビート判定を行い、必要に応じて START 信号やテンポ情報を同期制御部へ送信する流れを示している。

表 2.4 主要関数の仕様

クラス:: 関数名	引数	戻り値	処理内容
IMUReader::readIMU	なし	IMUData	IMU センサから 3 軸加速度および 3 軸角速度を取得する。
MotionFilter::removeGravity	IMUData data	IMUData	ハイパスフィルタを用いて加速度データから重力成分を除去する。
MotionFilter::calcMotion	IMUData data	float	3 軸加速度から合成加速度を算出する。
MotionFilter::smooth	float motion	float	移動平均により加速度データのノイズを低減する。
BeatDetector::detectPeak	float motion	bool	合成加速度から拍に相当するピークを検出する。
DownbeatDetector::detectDownbeat	IMUData data	bool	ジャイロデータからダウンビートであるかを判定する。
TempoCalculator::calcBPM	unsigned long interval	float	ピーク間隔から瞬時 BPM を算出する。
TempoCalculator::averageBPM	float bpm-List[]	float	1 小節分の BPM を平均化し、小節単位のテンポを算出する。
TempoCalculator::convertLevel	float bpm	int	算出した BPM を段階化してテンポレベルへ変換する。
SensorSender::sendStart	なし	なし	初回ダウンビート検出時に START 信号を同期制御ユニットへ送信する。
SensorSender::sendTempo	float bpm, int level	なし	算出した BPM およびテンポレベルを同期制御ユニットへ送信する。
SyncController::respondPolling	なし	Command	楽器制御ユニットからの要求に応じて開始指示やテンポ情報を返す。
SyncController::updateCommand	Command cmd	なし	センサ制御部から受信した開始状態およびテンポ情報を内部状態へ反映する。
InstrumentController::updateTempo	int bpm	なし	受信した BPM 値に基づき 1 拍あたりの待機時間を再計算する。
InstrumentController::sendFrequencyTrigger	なし	なし	楽曲データから周波数を取得し、Processing へシリアル送信する。
InstrumentController::drawFrogMatrix	bool isPlaying	なし	演奏状態に応じて LED マトリクス上の表示を更新する。
FrogServoController::moveFrog	int noteIndex	なし	指定された音階番号に対応するサーボモータを駆動する。
SerialReceiver::serialEvent	なし	なし	シリアルポートから周波数データを読み込み、数値へ変換する。
AudioPlayer::playFrequency	float hz	なし	受信した周波数に基づいて音声波形を生成し出力する。

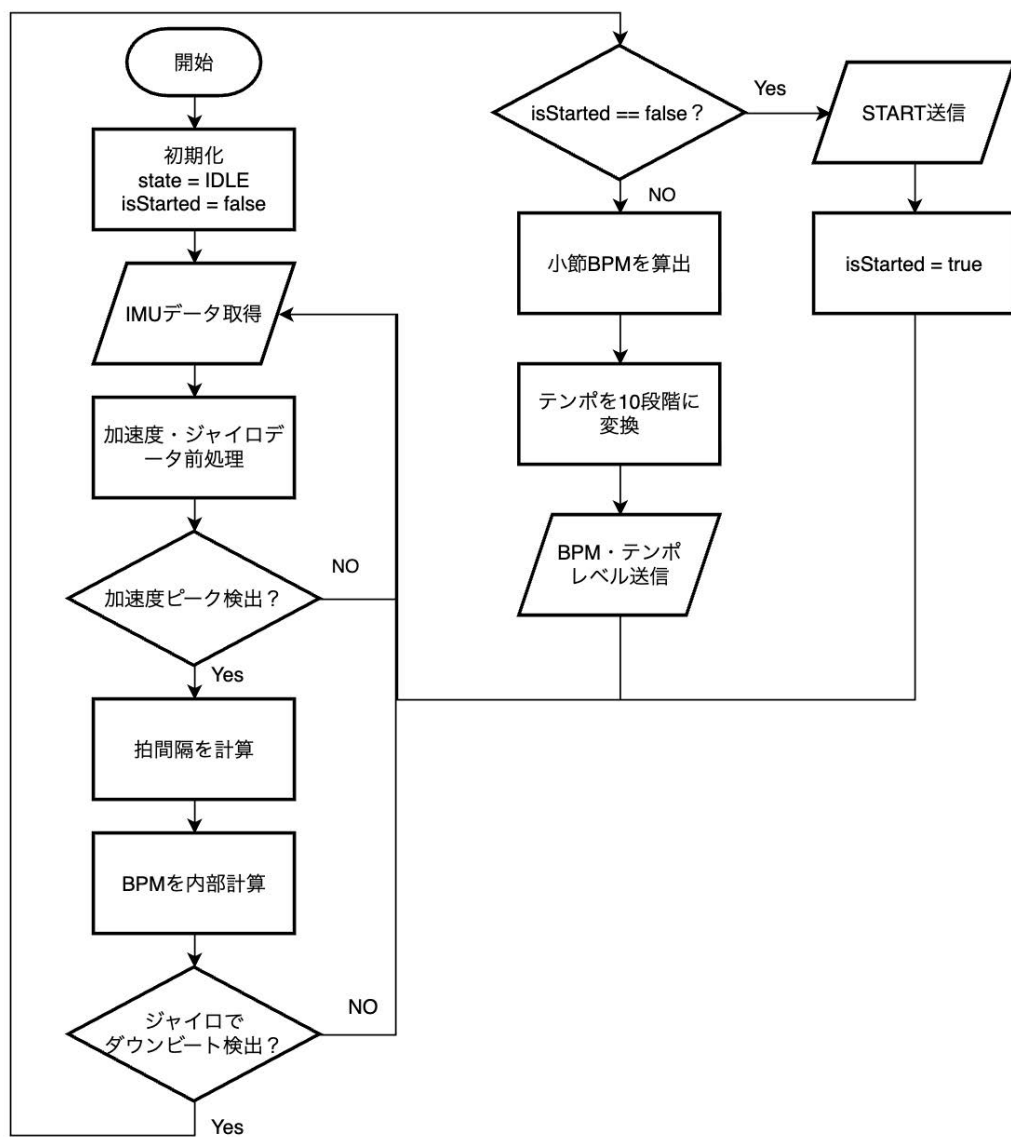


図 2.4 センサ制御部の処理フロー

図 2.4 は、センサ制御部が 1 周期の中でどの順番で処理を進めるかを示したフロー図である。IMU データの取得から前処理、ピーク検出、ダウンビート判定、START 送信、BPM 算出、テンポ送信へとつながる流れを示している。

2.4 処理フローの設計

本システムの処理フローは、センサ制御部、同期制御部、楽器制御部、音声合成部の順に連携して進行する。まず起動時に各 Arduino は初期化を行い、センサ制御部は待機状態、同期制御部は指示未発行状態、楽器制御部は演奏待機状態へ移行する。

その後、センサ制御部は IMU データを周期的に取得し、重力除去、平滑化、合成加速度算出を実施する。有効なピークとダウンビートが検出されると、初回のみ START 信号を同期制御部へ送信する。演奏開始後は、拍間隔から求めた BPM を一定周期で再計算し、テンポ情報として同期制御部へ送る。

同期制御部は、受信した START 信号とテンポ情報を内部状態として保持する。楽器制御ユニットからのポーリング要求に対して、現在の演奏開始状態と最新 BPM を返し、必要に応じて演奏開始指示またはテンポ更新指示を配信する。この構成により、複数ユニット間で通信状態を確認しながら同期動作を継続できる。

楽器制御ユニットは、演奏開始指示を受けると自ユニットの楽曲データを読み出し、BPM に応じた待機時間を計算して発音タイミングを管理する。発音時刻に達すると Processing へ周波数情報を送信し、該当音階に対応する LED 表示やサーボモータ演出を同時に実行する。テンポ更新指示を受けた場合は、次回以降の待機時間を再計算し、演奏を継続する。

Processing 側では、シリアル通信で受け取った周波数データを逐次解析し、発振器へ反映することでリアルタイムに音声波形を生成する。以上の処理を反復することで、ユーザの指揮動作に追従した演奏制御、音声出力、視覚演出が実現される。

処理フローをより細かく見ると、センサ制御部ではまず入力取得、次に前処理、その後に拍検出、ダウンビート判定、テンポ計算、送信判断という段階を順に通過する。この順序を固定することで、後段の判定が常に整形済みデータを対象に行えるようになり、処理の再現性を確保できる。

同期制御部では、受信した情報を即座に全ユニットへ無条件送信するのではなく、内部状態として保持してから各ユニットの要求に応答する形をとる。これにより、一時的に通信周期がずれても、各ユニットは次回の問い合わせ時点で最新状態へ復帰し

やすい。

楽器制御部では、演奏データの読み出し、待機時間計算、発音送信、演出出力の順に処理を進める。発音と演出を同一タイミングのイベントとして扱うことで、音だけ先行する、あるいは演出だけ遅れるといった視覚的な違和感を抑える。

例外的な状況として、センサ制御部が一時的に不安定な値を取得した場合には、直前値や平均化した BPM を用いて急激な変化を抑えることが望ましい。また、楽器制御部が一度テンポ更新を受け取れなかった場合でも、次回ポーリング時に最新テンポを再取得することで、長時間の不整合が残らない構成としている。

センサ制御部単体の処理フローでは、初期化、IMU 取得、前処理、ピーク検出、ダウンビート判定、START 送信、BPM 平均化、テンポ情報送信が順に並ぶ。全体の処理フローは、この流れに同期制御部と楽器制御部の処理を連結した構成である。

2.5 テストの設計

設計した各ユニットが要件を満たすことを確認するため、表 2.5 に示すテストを実施する。センサ制御、同期通信、楽器制御、音声出力、視覚演出の各観点で検証し、分散システムとしての整合性を確認する。

表 2.5 は、実装後に確認する検証項目を整理したものである。センサ取得の正しさ、通信の安定性、テンポ追従、視覚演出の同期を分けて確認できるようにしている。

テストは、まず単体レベルで各制御ユニットを個別に確認し、その後に結合レベルで全体動作を確認する手順を想定する。センサ制御部の単体試験では、シリアルモニタやログ出力を用いて取得値、ピーク検出結果、BPM 算出値を確認し、同期制御部の単体試験ではポーリング応答が想定どおりの状態遷移を返すかを検証する。

次に結合試験では、センサ制御部からの START 信号を起点として、同期制御部、楽器制御部、Processing が順に連携するかを確認する。この段階では、単に音が鳴ることだけでなく、どの楽器がどの順序で開始したか、テンポ変更がどの程度の遅れで反映されたか、視覚演出が発音と一致しているかといった複数観点で評価する必要がある。

展示システムとしての完成度を高めるためには、定量評価と定性評価の両方が重要である。定量評価としては、通信成功率、BPM 算出誤差、テンポ追従遅延などを計測する。定性評価としては、ユーザが自然に操作できるか、観客から見て指揮動作と演奏結果の対応が理解しやすいかを確認する。これらを総合して、本システムが技術

表 2.5 システムのテスト項目

テスト項目	検証内容
IMU データ取得テスト	IMU センサから 3 軸加速度および 3 軸角速度が安定して取得できるかを確認し、静止時と動作時で値が適切に変化することを確認する。
ピーク検出テスト	指揮棒を一定テンポで振った際に、加速度データから各拍に対応するピークが検出されるかを確認し、実際の振り回数と検出結果の差を評価する。
START 判定テスト	初回の有効なダウンビート検出時のみ START 信号が送信され、演奏開始後に再送されないことを確認する。
BPM 算出・テンポ追従テスト	指揮棒の往復周期に応じて BPM が算出され、テンポ変更時に各楽器制御ユニットの演奏速度が追従することを確認する。
テンポレベル変換テスト	算出された BPM が設計した段階値へ変換され、同期制御部へ一貫したテンポ情報として送られることを確認する。
通信・ポーリングテスト	同期制御ユニットと各楽器制御ユニット間でポーリング処理が正しく行われ、開始指示や更新指示が取りこぼしなく配信されることを確認する。
同時発音テスト	複数の楽器制御ユニットから同時にデータが送られた際、Processing 側で各音が正しく受信・再生されることを確認する。
視覚演出同期テスト	発音タイミングと同時に LED 表示およびサーボモータ駆動が行われ、音声出力とのずれが許容範囲内であることを確認する。
演奏順序テスト	輪唱の開始順序や担当パートが設計通りに制御され、各ユニットが正しい順番で演奏を開始することを確認する。

的にも体験的にも成立しているかを判断する.

通信信頼性の目安として, Arduino 間通信の成功率を 98%以上とする. 開始指示とテンポ更新指示が高い成功率で伝達されることを確認する. また, センサ入力時, 輪唱開始時, テンポ変更時などの条件別に失敗の傾向を確認する.